THE 11-STATE SQUARE-ROOT-OF-TWO MACHINE

A beginner-friendly reconstruction of a binary Turing machine that halts exactly when $\sqrt{2}$ is rational

Machine: 11 working states, alphabet {0, 1}, blank tape

The central idea

The machine runs two unary countdowns on one tape. One countdown marks the gaps between consecutive squares: 1, 3, 5, 7, … . The other marks the gaps between consecutive twice-squares: 2, 6, 10, 14, … . The machine halts only when the two event schedules coincide. Such a coincidence would give positive integers x and y with $x^2 = 2y^2$, which is impossible.

Prepared from the supplied transition table

November 2026

Reading note: This paper explains the supplied 11-state machine itself. It uses explicit tape configurations and transition traces, rather than assuming familiarity with unary counters or Turing-machine state tables.

1. The mathematical problem

A Turing machine can search for a counterexample to a mathematical statement. It cannot, in general, decide an infinite problem by checking a finite number of cases. But it can be designed so that it halts if it ever finds a witness. The 11-state machine studied here is such a search machine.

The question is whether $\sqrt{2}$ is rational. If $\sqrt{2} = x/y$ for positive integers x and y, then squaring both sides gives $x^2 = 2y^2$. Conversely, any positive integer solution of $x^2 = 2y^2$ gives the rational expression $x/y = \sqrt{2}$. Thus the machine can search for a solution of

$$x^2 = 2y^2, \qquad \text{with } x > 0 \text{ and } y > 0.$$

The machine does not store x and y as ordinary binary numerals, and it does not multiply them. Instead, it constructs two sequences of event times and looks for a time belonging to both sequences.

1.1 The square clock

The identity

$$1 + 3 + 5 + \cdots + (2x - 1) = x^2$$

shows that the squares can be generated by repeatedly adding consecutive odd numbers. Start with a countdown of 1. When it expires, the clock has reached 1^2. Reset the countdown to 3; the next expiry occurs three steps later, at time $4 = 2^2$. Then reset to 5, then 7, and so on.

Event number x	Period just used	Cumulative event time
1	1	$1 = 1^2$
2	3	$1 + 3 = 4 = 2^2$
3	5	$1 + 3 + 5 = 9 = 3^2$
4	7	$16 = 4^2$
5	9	$25 = 5^2$

1.2 The twice-square clock

Double every odd increment. Its periods are 2, 6, 10, 14, … . After y periods, the elapsed time is

$$2 + 6 + 10 + \cdots + (4y - 2) = 2y^2.$$

Event number y	Period just used	Cumulative event time
1	2	$2 = 2 \cdot 1^2$
2	6	$8 = 2 \cdot 2^2$
3	10	$18 = 2 \cdot 3^2$
4	14	$32 = 2 \cdot 4^2$
5	18	$50 = 2 \cdot 5^2$

What the machine searches for

The left clock rings at times 1, 4, 9, 16, 25, … . The right clock rings at times 2, 8, 18, 32, 50, … . A simultaneous ring at time T would mean $T = x^2 = 2y^2$.

2. The Turing-machine convention

The tape extends infinitely in both directions. Every cell initially contains 0, and the head begins at cell 0 in state A. The tape alphabet is exactly $\{0, 1\}$. The symbol 0 serves both as the untouched blank and as a temporary separator or moving gap; the machine distinguishes these zeros by their position and by the state of its head.

An instruction has three characters. For example, 1LC means: write 1 in the cell being read, move one cell Left, and enter state C. Similarly, 0RZ means: write 0, move Right, and enter the halting state Z.

2.1 The complete transition table

State	On 0	On 1	Main role (informal)
A	1LC	1LA	Extend the far-left end; otherwise scan left.
B	0LF	1LB	Scan left to the central 0, then return to F.
C	1RD	1LC	Scan left across 1s; on 0, fill it and turn right.
D	1LK	0RE	On 0, fill it and start K; on 1, erase it.
E	0RG	1RE	Scan right across 1s; on 0, hand off to G.
F	1LA	1LC	Test the left countdown; on 0, trigger the reset.
G	1LH	1RG	Scan right across 1s; on 0, fill a gap or setup blank.
H	1LB	0LI	On 1, erase it; on 0, fill the setup blank.
I	0LJ	1LB	On 1, return; on 0, enter J.
J	0RZ	0RC	On 0, halt; on 1, begin the next interval.
K	1LC	1RK	Scan right to the outer blank and append one mark.

These role descriptions are summaries, not replacements for the table. Some states have different jobs on their two input symbols. That is intentional: the machine reuses a state when the tape invariant guarantees which symbol will be encountered.

2.2 The supplied compact string

```
1LC1LA_0LF1LB_1RD1LC_1LK0RE_0RG1RE_1LA1LC_1LH1RG_1LB0LI_0LJ1LB_0RZ0RC_1LC1RK
```

The underscores separate the eleven state blocks A through K. Inside each block, the first three-character instruction is for reading 0, and the second is for reading 1. The machine has no outgoing transition from Z.

3. How the tape stores the clocks

A block of n consecutive 1s represents the number n. This is called unary notation: the number is represented by repeating one mark n times. A moving zero gap divides a block into two parts. The number of 1s on one side of the gap can be read as a countdown.

3.1 A checkpoint: state J

The most useful configurations are the moments when the machine enters state J. Put the central separator at tape position 0. At such a checkpoint, the tape has this shape:

```
... 1^(p−r)  0  1^r  [0]  0  1^q  0 ...
                  left   centre  right
                   gap            gap
```

The head is on position −1, the final cell of the inner left-hand run 1^r, and the current state is J. The symbols have these meanings:

Quantity	Tape meaning	Arithmetic meaning
p	Total number of 1s in the left clock block	Current square-clock period
r	Number of 1s between the left gap and the centre	Time remaining until the next square event
q	Number of 1s to the right of the right gap	Most recently completed twice-square period

The two zeros next to one another are important. The first is the central separator. The second is the right gap. If the left countdown is positive, cell −1 contains 1, so J takes its 1-branch. If the countdown is zero, the left gap has moved all the way to cell −1; J reads 0 and halts.

The checkpoint invariant

At the y-th visit to J, let x be the least integer with $x^2 \geq 2y^2$. Then $p = 2x - 1$, $r = x^2 - 2y^2$, and $q = 4y - 2$. Thus the machine halts at J exactly when r = 0, which is exactly the equation $x^2 = 2y^2$.

3.2 Reading a small example

```
... 1  0  1  1  0  0  1  1 ...
   -4 -3 -2 -1  0  1  2  3      (cell positions)
```

This is the first J checkpoint. There are three 1s in the left block: one before the left gap and two after it. Hence p = 3 and r = 2. There are two 1s after the right gap, so q = 2. The arithmetic values are x = 2 and y = 1: $2^2 - 2 \cdot 1^2 = 2$, exactly the left countdown r.

4. How the machine starts on a blank tape

The machine does not begin in a J checkpoint. It first builds the initial tape pattern using ordinary working states. This startup phase takes 29 transitions. By its end, the machine has reached the first useful checkpoint, corresponding to the first twice-square event, time 2.

Here is the complete startup trace. “Head” is the cell being read before the instruction. The entry such as 1LC tells exactly what is written, which way the head moves, and the next state.

Step	State	Head cell	Instruction	Next configuration
1	A	0	1LC	C @ −1
2	C	−1	1RD	D @ 0
3	D	0	0RE	E @ 1
4	E	1	0RG	G @ 2
5	G	2	1LH	H @ 1
6	H	1	1LB	B @ 0
7	B	0	0LF	F @ −1
8	F	−1	1LC	C @ −2
9	C	−2	1RD	D @ −1
10	D	−1	0RE	E @ 0
11	E	0	0RG	G @ 1
12	G	1	1RG	G @ 2
13	G	2	1RG	G @ 3
14	G	3	1LH	H @ 2
15	H	2	0LI	I @ 1
16	I	1	1LB	B @ 0
17	B	0	0LF	F @ −1
18	F	−1	1LA	A @ −2
19	A	−2	1LA	A @ −3
20	A	−3	1LC	C @ −4
21	C	−4	1RD	D @ −3
22	D	−3	0RE	E @ −2
23	E	−2	1RE	E @ −1
24	E	−1	1RE	E @ 0
25	E	0	0RG	G @ 1
26	G	1	1RG	G @ 2
27	G	2	1LH	H @ 1
28	H	1	0LI	I @ 0
29	I	0	0LJ	J @ −1

The first important reset is triggered at step 18: F reads 0 and enters A. This detects that the left clock's first event, at clock time t = 1, has just occurred; the reset prepares the next odd period while the computation proceeds toward t = 2. At step 29, I reads the central zero and enters J, recording the first twice-square event at t = 2. The startup phase therefore incorporates the early events at times 1 and 2 before the regular J-to-J routine begins.

5. What happens when J reads 1?

Suppose J is reached with r > 0. It reads 1 at cell −1, so the machine cannot halt. The instruction in J is 0RC: it temporarily erases that final 1 and moves right into state C. The temporary erasure is repaired during the setup. The purpose of the setup is to turn the right block of q ones into a countdown of q + 4 ones.

5.1 The setup, one stage at a time

State	Read	Instruction	Effect
J	1	0RC	Temporarily erase cell −1; move to the centre
C	0	1RD	Fill the central zero; move right
D	0	1LK	Fill the right gap; begin the K scan
K	1	1RK	Scan right across the existing right block
K	0	1LC	Append a 1 at its far end; return left in C
C	1	1LC	Scan left across the joined run
C	0	1RD	Restore the temporary zero at −1
D	1	0RE	Restore the central separator at 0
E	1	1RE	Scan right to the first blank
E	0	0RG	Start G one cell beyond the block
G	0	1LH	Write the first new outer 1
H	0	1LB	Fill the preceding blank and return left
B	1	1LB	Return to the central separator
B	0	0LF	Enter F at cell −1

The two branches C and D temporarily join the central separator and the right-hand block into one run. K extends that run by one. The later C branch restores the erased cell at −1, while D creates the central zero again. E, G, and H then add two more 1s at the outer end. The initial right block had q ones, so the result has q + 4 ones between the centre and the new right gap.

```
At the next F boundary:   ... left clock ... 0 1^(q+4) 0 ...
                                  centre          countdown gap
```

Why four?

The next twice-square period after $2y^2$ is $2(y+1)^2 - 2y^2 = 4y + 2$. At the J checkpoint $q = 4y - 2$, so $q + 4 = 4y + 2$. The setup creates exactly the number of marks needed for the next right-clock interval.

6. The ordinary sweep: one unit of both clocks

After setup, the machine is in state F with its head at cell −1. Between the centre and the right gap there are s = q + 4 ones. In an ordinary sweep (the F-on-1 case), the left gap moves one cell to the right and the right gap moves one cell to the left. The two gaps therefore move toward their respective countdown endpoints. When F instead reads 0, the special left-period reset described in Section 7 replaces the ordinary left-gap move; the right-hand part of the sweep still advances the right clock by one unit.

6.1 The sequence of states

State	Action	Why it matters
F	On 1, keep it and move left to C	The left countdown has not expired
C	Move left across 1s; fill the left gap on 0	The left gap shifts one cell right
D	Erase the next inner 1 and move right to E	Completes the left-gap shift
E	Move right across 1s; preserve the central 0	Finds the right-hand side
G	Move right across 1s; fill the right gap on 0	Moves the right gap one cell left
H	Erase the preceding 1; move left to I	Creates the new right gap
I	If 1, go to B; if 0, go to J	Detects the right-clock event
B	Move left across 1s; stop at the central 0	Returns to F for the next sweep

The two important tests are F and I. F reads the last cell of the left countdown. I reads the cell immediately to the left of the new right gap. If I sees 1, the right countdown still has time remaining and B returns the head to F. If I sees the central 0, the right gap has reached the centre and the machine enters J.

6.2 A literal example: the first ordinary sweep

After the J-to-F setup for y = 1, the machine is at F, head −1. The active tape pattern is:

```
position:  −4  −3  −2  −1   0   1   2   3   4   5   6   7
symbol:     1   0   1   1   0   1   1   1   1   1   1   0
                           F               right gap
```

The left gap is at −3 and there are two 1s between it and the centre, so r = 2. The right gap is at 7, with six 1s between the centre and the gap, so s = 6. The transition trace is:

Steps	State/head	Instruction(s)	Effect
56	F @ −1	`1LC`	Move left; no left ring
57	C @ −2	`1LC`	Continue toward the left gap
58	C @ −3	`1RD`	Fill the left gap
59	D @ −2	`0RE`	Erase next 1; left countdown becomes 1
60–61	E @ −1, 0	1RE, 0RG	Cross to the centre and enter G
62–68	G @ 1…7	1RG × 6, 1LH	Fill the right gap
69	H @ 6	`0LI`	Erase the preceding 1
70	I @ 5	`1LB`	Right countdown not yet exhausted
71–75	B @ 4…0	1LB × 4, 0LF	Return to F

At the next F boundary (step 76), the left gap has moved from −3 to −2, leaving one inner 1. The right gap has moved from 7 to 6, leaving five inner 1s. One sweep has decreased both countdowns by one.

7. What happens when the left countdown reaches zero?

Eventually F is reached with a 0 beneath the head. This means the left gap has moved all the way to the cell next to the centre. F does not continue through the ordinary C branch. Instead, it uses its 0-instruction, 1LA, and enters state A.

7.1 The reset mechanism

When F reads 0, it first fills the exhausted gap at cell −1 and enters A. A scans left across the existing block. At the first blank cell just outside that block, A writes a 1 and moves one more cell left into C. C writes a 1 in that farther-out blank cell and moves right into D. D then erases the 1 that A wrote, leaving a zero gap between the new outer 1 (written by C) and the old block. Thus the combined F–A–C–D routine fills the old inner gap, adds one new outer 1, and relocates the gap to the outside edge. The total number of 1s rises by two.

```
                    Before:  ... 1^p  0_gap  0_centre
                  F fills the old gap; then F–A–C–D leave:
                 After:   ... 1  0_new gap  1^(p+1)  0_centre
The new outer 1 is written by C. The inner run has p+1 marks: the p old marks plus the
                              cell that F just filled.
```

The total number of 1s in the left clock has increased from p to p + 2. If the old period was $p = 2x - 1$, the new period is $p + 2 = 2(x + 1) - 1$, exactly the next odd period needed to generate the next square.

There is a one-step bookkeeping detail. The reset routine is part of the sweep that detected the expired countdown. By the time the machine returns to F, one unit of the new period has already been used. The number of 1s still remaining on the inner side is therefore p + 1, not p + 2. This is precisely what a countdown should show one step after the square event.

7.2 A concrete reset in the first interval

In the first interval, the machine reaches F at step 92 with a 0 at cell −1. The old left period is p = 3 and the countdown has just reached zero. F fills cell −1 and enters A. A scans across the three old marks at −2, −3, and −4, then writes 1 at −5. C writes 1 at −6. D erases the A-written mark at −5, leaving a new gap there and the new outer mark at −6.

```
                Old period: p = 3
              New period: p + 2 = 5
          Countdown on return to F: 4
Next square interval: 5 ticks in total
```

This matches the square sequence: after the event at $2^2 = 4$, the next square is $3^2 = 9$, five time units later. The machine has extended the left unary block by exactly the correct amount.

8. The arithmetic invariant and the J-to-J cycle

We can now connect the tape operations to the two arithmetic clocks. Suppose the machine enters J for the y-th time in the regular sequence, and let x be the least integer satisfying $x^2 \geq 2y^2$. The checkpoint values are

$$p = 2x - 1, \qquad r = x^2 - 2y^2, \qquad q = 4y - 2.$$

The state J checks r. If r = 0, the left square clock and right twice-square clock have rung at the same time. If r > 0, setup prepares the next right-clock countdown of q + 4 = 4y + 2. The machine then performs exactly q + 4 ordinary sweeps. The right gap shifts one cell toward the centre at each sweep; at the last sweep, I reads the central zero and sends the machine into J again.

Keep two notions of time separate. A machine transition is one microscopic read–write–move instruction; the counts 29, 155, and 439 in the checkpoint table count these instructions. A clock tick is one completed F-to-F sweep. The first J checkpoint represents clock time t = 2, and each later J-to-J interval consists of q+4 clock ticks, even though each tick takes a variable number of machine transitions.

8.1 Why the next checkpoint has the right values

The time between consecutive right-clock events is

$$2(y + 1)^2 - 2y^2 = 4y + 2 = q + 4.$$

Each sweep advances one unit of this interval. On the left, if F reads 1, C and D move the left gap one cell toward the centre, reducing the countdown by one. If F reads 0, the exceptional reset sweep begins: F fills the exhausted gap, then A–C–D move the gap to a new outer position. The combined F–A–C–D operation increases the period from p to p+2; because this sweep is already the first unit of the new period, p+1 marks remain on the inner side of the new gap. Thus the left-hand tape continues to track the next square event. After q+4 sweeps, the clock time has advanced from $2y^2$ to $2(y+1)^2$, so at the next J boundary

$$q_{next} = q + 4 = 4(y + 1) - 2,$$
$$p_{next} = 2x_{next} - 1,$$
$$r_{next} = x_{next}^2 - 2(y + 1)^2,$$
$$\text{where } x_{next} \text{ is the least integer with } x_{next}^2 \geq 2(y + 1)^2.$$

This is the induction step. The startup phase supplies the base case at the first J checkpoint: y = 1, x = 2, p = 3, r = 2, and q = 2.

Worked first interval. At the first J checkpoint, t = 2 and (p,r,q) = (3,2,2). Setup makes the right countdown q+4 = 6. The sweeps for t = 3 and t = 4 reduce the left countdown 2 → 1 → 0. The sweep for t = 5 begins with F reading 0, so the reset changes the left period from 3 to 5 and leaves 4 units in its new countdown. The sweeps for t = 6, 7, and 8 reduce that countdown to 1. The next J checkpoint is therefore at t = 8 with (p,r,q) = (5,1,6), as shown in the table.

8.2 Checkpoints from the actual machine run

y	Step at J	x	p	r	q	Next sweeps q+4
1	29	2	3	2	2	6
2	155	3	5	1	6	10
3	439	5	9	7	10	14
4	931	6	11	4	14	18
5	1,651	8	15	14	18	22
6	2,761	9	17	9	22	26
7	4,137	10	19	2	26	30
8	6,057	12	23	16	30	34

For example, at y = 3 the right clock has just rung at time 18. The least x with $x^2 \geq 18$ is x = 5. Hence p = 9 and r = 25 − 18 = 7. The right period just completed is q = 10, so the next interval takes q + 4 = 14 sweeps. The next J checkpoint is y = 4, at time 32; there x = 6 and r = 36 − 32 = 4.

9. Why the machine halts exactly when √2 is rational

The checkpoint invariant proves the connection between the machine and the equation. At a J checkpoint, the head is at cell −1. The tape symbol there is 1 exactly when r > 0, and it is 0 exactly when r = 0. State J has the instructions

```
J on 0:  0RZ       (halt)
J on 1:  0RC      (continue)
```

Thus the machine halts if and only if some checkpoint has r = 0. By the invariant, this happens if and only if

$x^2 - 2y^2 = 0$, equivalently $x^2 = 2y^2$,

for positive integers x and y. This is equivalent to √2 being rational.

9.1 The parity proof that no halt occurs

Assume, for contradiction, that positive integers x and y satisfy $x^2 = 2y^2$. Choose such a solution with x + y as small as possible. The equation says x^2 is even. An odd integer has an odd square, so x must be even. Write x = 2a. Substitution gives

$(2a)^2 = 2y^2$, so $4a^2 = 2y^2$, hence $y^2 = 2a^2$.

The same argument now shows that y is even. Write y = 2b. Then

$(2b)^2 = 2a^2$, so $4b^2 = 2a^2$, hence $a^2 = 2b^2$.

Therefore (a,b) is another positive integer solution of the same equation. But a = x/2 and b = y/2, so a + b = (x + y)/2 < x + y. This contradicts the minimal choice of x + y. No positive integer solution exists.

Conclusion

The mathematical equation has no positive integer solution. The machine's only route into Z is the J-on-0 transition, and that transition is possible only at a checkpoint satisfying $x^2 = 2y^2$. Therefore the blank-tape computation never halts.

10. What has been checked, and what the proof relies on

There are two different kinds of evidence in a machine analysis. A simulation checks that the transition table behaves as expected for a large finite number of steps. A mathematical invariant explains why the same pattern continues for every later step. The second is what proves nonhalting.

10.1 Direct computational checks

The supplied 11-state string was simulated from the all-zero tape using the convention stated in Section 2. The simulator reached 59 entries into J in the first 2,000,000 transitions. At every such checkpoint it had the required head position and tape shape, and the decoded values agreed with $p = 2x - 1$, $r = x^2 - 2y^2$, and $q = 4y - 2$. No entry into J read 0 during that run.

The first eight checkpoint indices and transition counts are listed in Section 8. The machine's first J entry occurs after 29 transitions; the next ones occur after 155, 439, 931, 1,651, 2,761, 4,137, and 6,057 transitions. The number of ordinary sweeps from one J checkpoint to the next is $q + 4$, namely 6, 10, 14, 18, ...

10.2 What establishes the infinite claim

The simulation alone does not prove that the machine runs forever. The proof is the combination of four facts:

- The blank-tape startup reaches the first J checkpoint with $(p,r,q) = (3,2,2)$.
- The J-on-1 setup leaves the entire left block unchanged. It turns the q marks to the right of the old right gap into $q+4$ countdown marks between the central separator and a new right gap.
- Each F-to-F sweep advances the clock time by one. If F reads 1, C and D move the left gap one cell inward and reduce its countdown by one. If F reads 0, the combined F–A–C–D reset increases the left period by two and starts the next countdown; the right-hand part of that same sweep still advances the right clock by one.
- After exactly $q+4$ sweeps, the right gap has moved to cell 1, immediately to the right of the central separator at cell 0. On that last sweep, I reads the central 0 and sends the head into J again, with the checkpoint values for $y+1$.

These facts give an induction over successive J checkpoints. The parity proof then rules out $r = 0$ at every checkpoint. This is the mathematical reason the computation cannot halt.

10.3 Scope and formalization status

This paper gives a hand-auditable reconstruction of the supplied transition table and its macro-level tape operations. It is not a proof-assistant development, and the state-by-state sweep argument has not been encoded in a kernel-checked formal system here. For proof-theoretic work, the next useful step would be to formalize the checkpoint and sweep lemmas in a weak arithmetic such as $I\Delta_0$, including explicit definitions of the tape configuration and the transition relation.

Appendix A. Quick reference

A.1 The arithmetic schedules

```
Square clock:       1, 4, 9, 16, 25, 36, ...
Increments:          1, 3, 5,  7,  9, 11, ...

Twice-square clock:  2, 8, 18, 32, 50, 72, ...
Increments:          2, 6, 10, 14, 18, 22, ...
```

A.2 The J-checkpoint invariant

```
Tape:  ... 1^(p−r) 0 1^r [0] 0 1^q 0 ...
Head: cell −1, state J

p = 2x − 1
r = x² − 2y²
q = 4y − 2
x = least integer with x² ≥ 2y²
```

A.3 The main state cycle

```
F → C → D → E → G → H → I → B → F

F on 0 takes the reset branch through A.
I on 0 enters J.
J on 0 halts; J on 1 performs setup through C, D, K, C, D, E, G, H, B.
```

A.4 The machine string

```
1LC1LA_0LF1LB_1RD1LC_1LK0RE_0RG1RE_1LA1LC_1LH1RG_1LB0LI_0LJ1LB_0RZ0RC_1LC1RK
```

Instruction order: states A, B, C, D, E, F, G, H, I, J, K. Within each state block, the first instruction is for symbol 0 and the second is for symbol 1.